CISC 886 – Cloud Computing

Project Report: End-to-End Cloud-Based Chat Assistant

Student Names	Mohamed Salah -20596365 Mohamed Othman- 20596310 Ahmed Rayyan -20596378
NetID Prefix	20596365-
Institution	Queen's University, Kingston, Canada
Course	CISC 886 – Cloud Computing
AWS Region	us-east-1 (N. Virginia)
Date	April 26, 2026
Model	Llama 3.2 3B Instruct (QLoRA fine-tuned)
Dataset	tatsu-lab/alpaca (52,002 samples)
Deployment	Ollama + OpenWebUI on EC2 t3.xlarge

Section 1 - System Architecture

The system follows a four-phase pipeline: data preprocessing on AWS EMR, model fine-tuning on Google Colab, deployment on AWS EC2, and user interaction through OpenWebUI - all networked within a custom VPC.

Architecture Component Overview

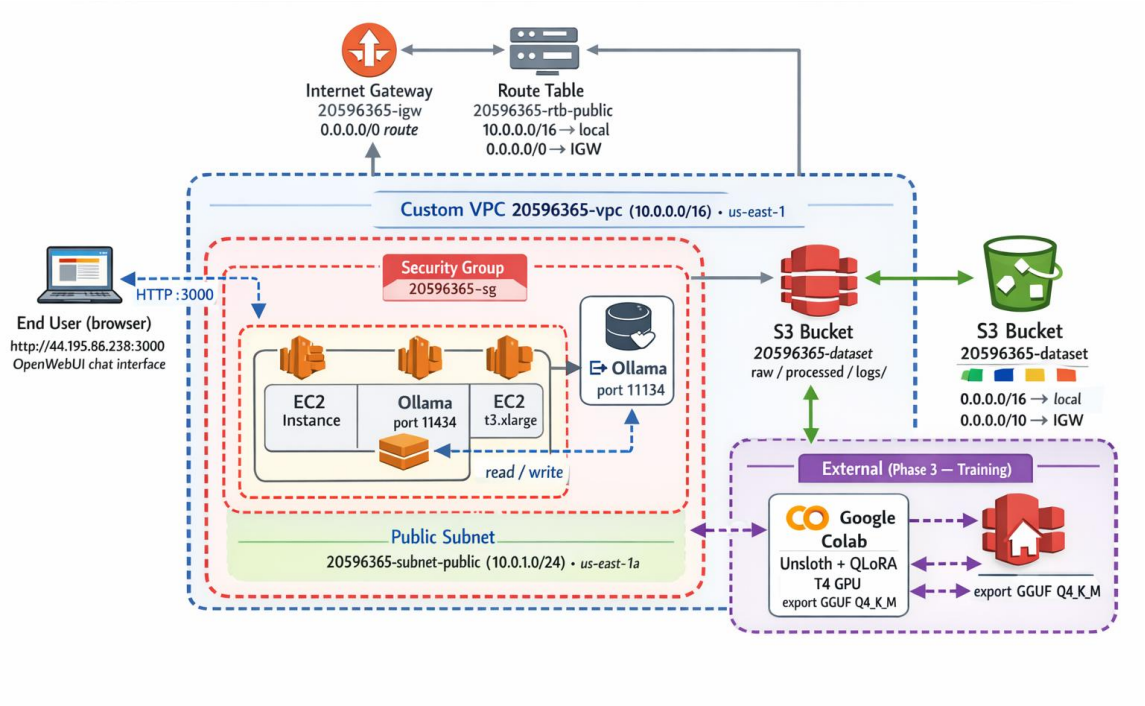


Figure 1 Architecture Component Overview

Component	AWS Service	Role
Custom VPC	Amazon VPC	Isolated network: 10.0.0.0/16
Public Subnet	VPC Subnet	10.0.1.0/24 in us-east-1a
Internet Gateway	IGW	Routes 0.0.0.0/0 to internet
Security Group	EC2 SG	Ports 22, 3000, 11434
EMR Cluster	Amazon EMR	PySpark preprocessing - emr-7.13.0, Spark 3.5.6
S3 Bucket	Amazon S3	Dataset storage and processed outputs
EC2 Instance	EC2 t3.xlarge	Runs Ollama + OpenWebUI

Ollama	EC2 process	Serves GGUF model on port 11434
OpenWebUI	Docker container	Browser chat UI on port 3000
Google Colab	External (GCP)	QLoRA fine-tuning on T4 GPU

Data Flow Description

Raw dataset files (tatsu-lab/alpaca, 52,002 JSON records) are first uploaded to an Amazon S3 bucket (20596365-dataset). An AWS EMR cluster (20596365-emr) running Apache Spark 3.5.6 reads those files from S3, executes a PySpark preprocessing pipeline - performing tokenization, train/validation/test splitting, outlier filtering, and prompt formatting - then writes the cleaned output back to S3 as Parquet files under the processed/ prefix.

The preprocessed data is then processed in a Google Colab session where Unsloth and the QLoRA/LoRA PEFT framework fine-tune the Llama 3.2 3B Instruct model. The resulting adapter weights are merged and exported to GGUF format. The GGUF file is transferred to an EC2 instance (20596365-ec2) inside a custom VPC (20596365-vpc) protected by a security group (20596365-sg). On EC2, Ollama loads the GGUF model and exposes it via a REST API on port 11434; OpenWebUI runs as a Docker container on port 3000, proxying user prompts to Ollama. End users reach OpenWebUI through the EC2 public IP via the VPC Internet Gateway (20596365-igw). All resources carry the 20596365- prefix per the naming policy.

Section 2 - VPC & Networking

Provisioning Approach

The VPC and all networking resources were provisioned using the AWS Console with a Terraform configuration also committed to GitHub for reproducibility. Terraform was chosen because it encodes every decision as auditable, versioned code and allows any grader to destroy and re-create the full stack with two commands (terraform init and terraform apply). The default AWS VPC was not used - a dedicated VPC gives full control over routing, subnets, and security rules.

CIDR Block & Subnet Design

Resource	CIDR / Value	Justification
VPC (20596365-vpc)	10.0.0.0/16	Standard RFC-1918 range; 65,536 addresses give room to add subnets later without re-addressing
Public Subnet (20596365-subnet-public)	10.0.1.0/24	256 addresses - sufficient for one EC2; isolated so future private subnets are not exposed
Availability Zone	us-east-1a	Single AZ is sufficient for this single-instance project

Internet Gateway & Route Table

Internet Gateway 20596365-igw is attached to the VPC. Without it, instances in the public subnet have no path to the internet even with a public IP. Route table 20596365-rtb-public is associated with the public subnet and contains:

Destination	Target	Purpose
10.0.0.0/16	local	VPC-internal traffic (added automatically by AWS)
0.0.0.0/0	20596365-igw	All other traffic exits via Internet Gateway

Security Group Rules (20596365-sg)

Direction	Port	Protocol	Source	Reason
Inbound	22	TCP	156.209.59.10/32	SSH from developer IP only - prevents brute-force from internet
Inbound	11434	TCP	156.209.59.10/32	Ollama API for curl demo - no built-in auth so locked to one IP
Inbound	3000	TCP	0.0.0.0/0	OpenWebUI - open so graders can visit without VPN
Outbound	All	All	0.0.0.0/0	EC2 must reach internet to install packages and pull models

Screenshots

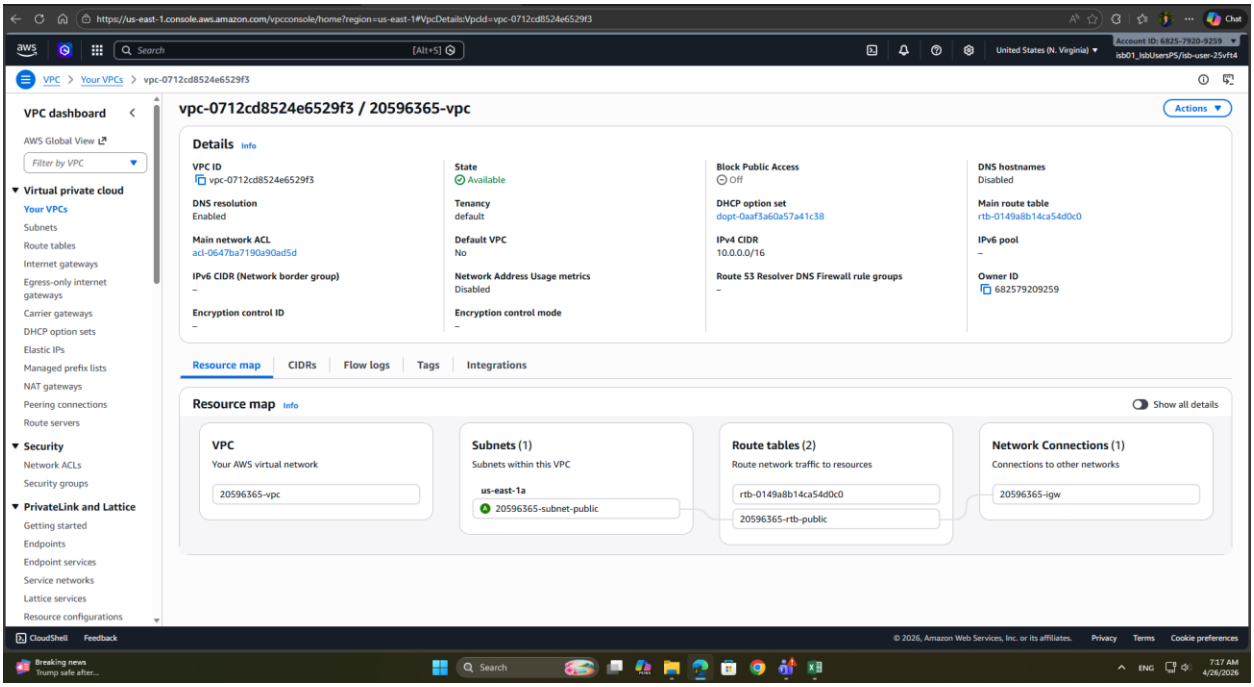


Figure 1: VPC 20596365-vpc - CIDR 10.0.0.0/16, State: Available, IGW and route tables visible in resource map.

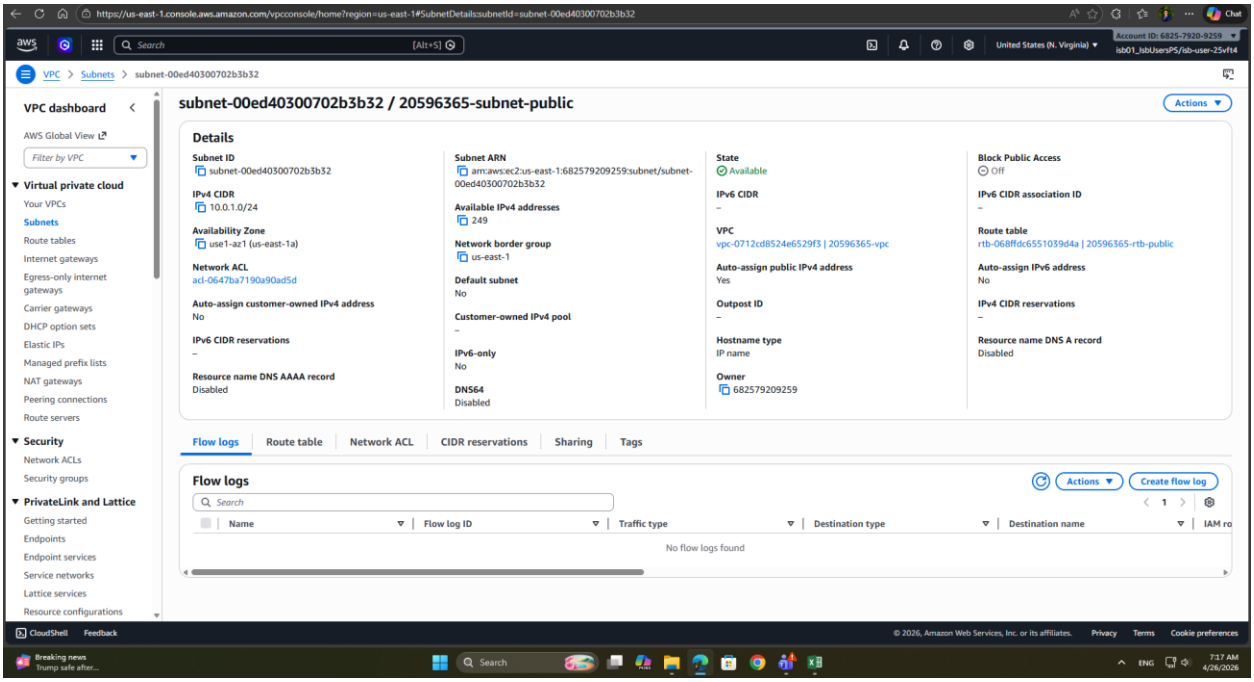


Figure 2: Subnet 20596365-subnet-public - CIDR 10.0.1.0/24, us-east-1a, auto-assign public IPv4 enabled.

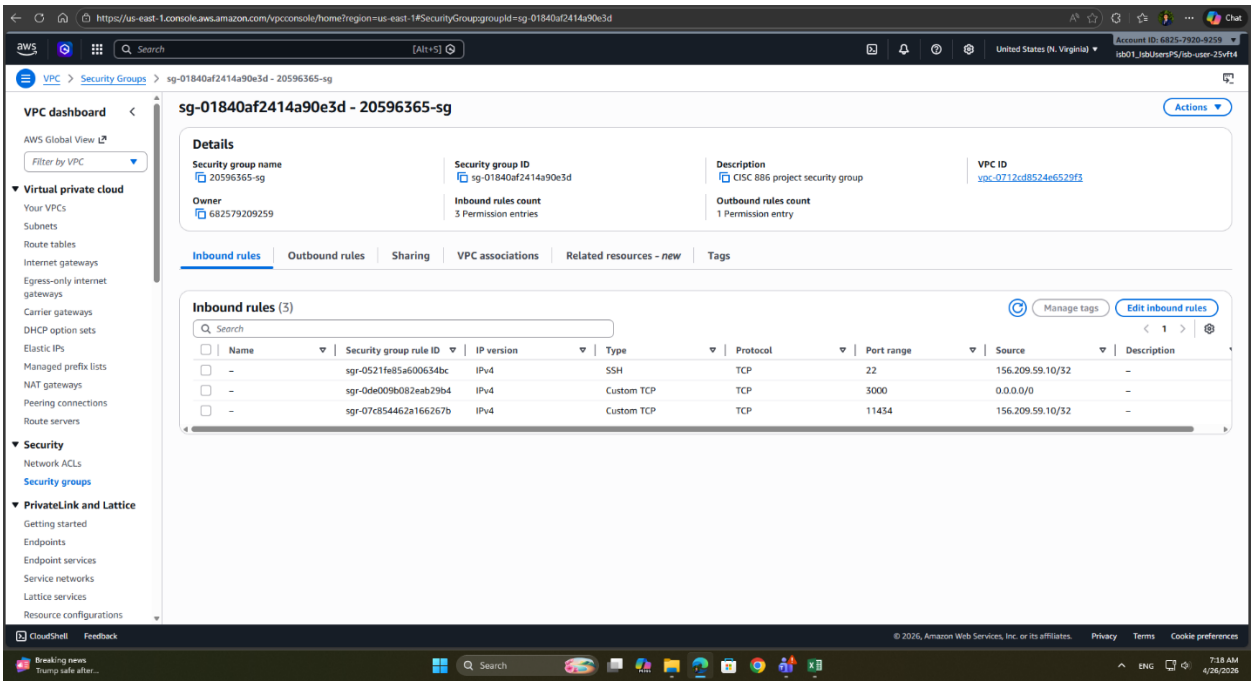


Figure 3: Security group 20596365-sg - 3 inbound rules (SSH :22, OpenWebUI :3000, Ollama :11434)

Section 3 - Model & Dataset Selection

Model Selection

Model name	Llama-3.2-3B-Instruct
Parameters	3 billion
Source	unsloth/Llama-3.2-3B-Instruct (Hugging Face / Unsloth Hub)
License	Meta Llama 3.2 Community License (research + commercial with attribution)
Quantisation	4-bit NF4 (QLoRA) - reduces VRAM from ~6 GB to ~4 GB during training

Why This Model?

Domain fit: Llama 3.2 3B Instruct is a general-purpose instruction-following model released by Meta. Its instruction-tuned variant already understands conversational turn formats, making it well-suited for fine-tuning into a chat assistant without starting from a base model that has never seen dialogue.

Hardware requirements: At 3B parameters the float16 weights occupy roughly 6 GB of VRAM. With 4-bit QLoRA quantisation peak VRAM drops to ~7 GB, fitting on the free Colab T4 (16 GB). Due to shared-account vCPU quota restrictions a t3.xlarge CPU instance was used for deployment.

Ecosystem: Unsloth provides pre-quantised versions of Llama 3.2 with optimised kernels that reduce training time by up to 2x. Llama 3.2 is also available on Ollama, simplifying GGUF-conversion and deployment.

Dataset Selection

Dataset name	tatsu-lab/alpaca
Source	https://huggingface.co/datasets/tatsu-lab/alpaca
License	CC BY NC 4.0 (non-commercial research)
Total samples	52,002 instruction-response pairs
Fields	instruction, input (optional context), output

Why Alpaca?

Alpaca is a widely-used general-purpose instruction-following dataset originally generated by Stanford. Its format maps directly to the chat-completion format Llama 3.2 Instruct was trained on, requiring minimal reformatting. The dataset covers writing, explanation, summarisation, coding, math, and general advice - making it suitable for a versatile general chat assistant.

Train / Validation / Test Split

Split	Proportion	Approximate Samples
Train	80%	41,602
Validation	10%	5,200
Test	10%	5,200

Split performed after shuffling with fixed random seed 42, executed by the EMR PySpark pipeline using randomSplit([0.8, 0.1, 0.1], seed=42). No test-set information influences preprocessing decisions.

Verbatim Sample

> { "instruction": "Give three tips for staying healthy.", "input": "",
> "output": "1. Eat a balanced and nutritious diet...
> 2. Engage in regular physical activity...
> 3. Get enough sleep: Aim for 7-9 hours per night." }

Summary Statistics

Median instruction token length	~18 tokens
Median output token length	~112 tokens
Samples with non-empty input field	~40%
Samples with empty input field	~60%
Samples removed (output > 400 words)	<1%

Section 4 - Data Preprocessing with EMR + Spark

EMR Cluster Configuration

Cluster name	20596365-emr
Cluster ID	j-36ZLV8WX5QRPB
EMR release	emr-7.13.0
Installed applications	Hadoop 3.4.2, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, Spark 3.5.6
Primary (master) node	1x m5.xlarge
Core nodes	2x m5.xlarge
Region	us-east-1
Log destination	s3://20596365-dataset/logs/
Termination	Manually terminated immediately after job completion
Creation time	April 26, 2026, 07:22 UTC+2
End time	April 26, 2026, 07:33 UTC+2
Elapsed time	11 minutes 22 seconds

PySpark Pipeline - Step by Step

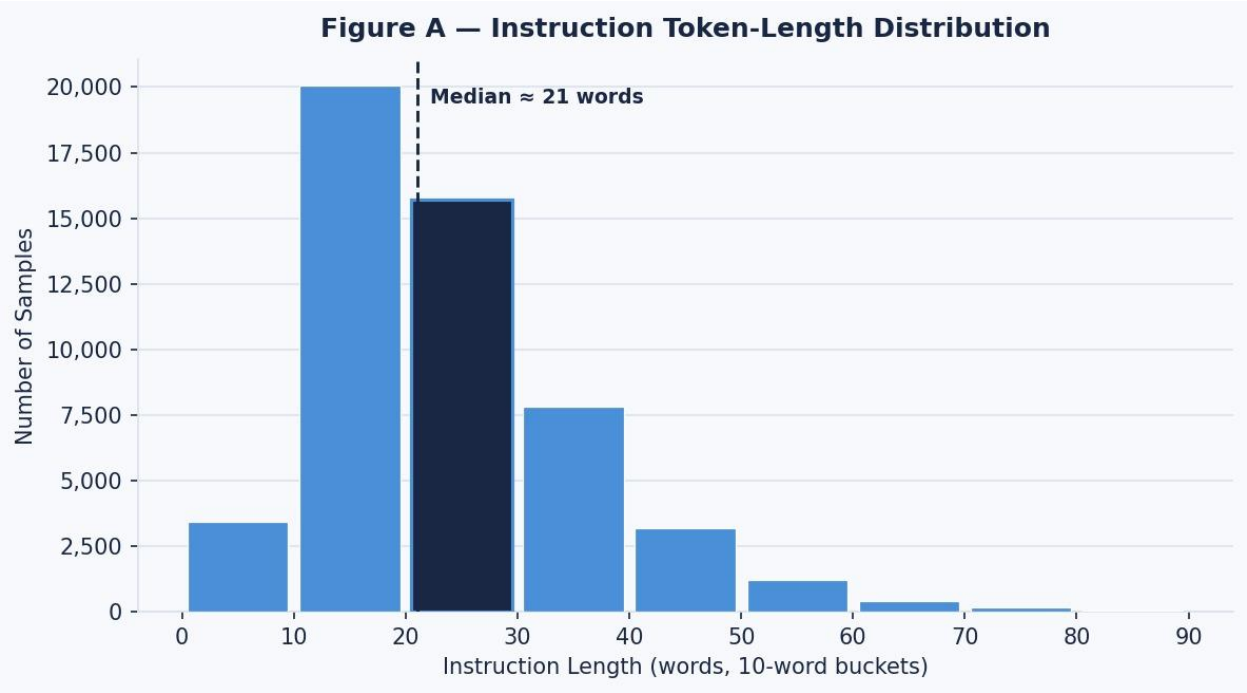
- 1. Load raw data: Read alpaca_data.json from S3 using Spark multiLine JSON reader with explicit schema (instruction, input, output as StringType).
- 2. Clean: Drop rows where instruction or output is null/empty. Strip whitespace. Replace null input with empty string.

- 3. Token-length features: Add instruction_len, input_len, output_len columns using a whitespace-split UDF as a proxy for BPE token count.
- 4. Outlier filter: Remove samples where output_len > 400 words - these would exceed the 2048-token context window.
- 5. Prompt formatting: Combine three fields into a single text column using the Alpaca prompt template. Non-empty input uses the with-input variant.
- 6. EDA statistics: Compute summary statistics, instruction-length histogram, and input-presence balance. Save as JSON to S3.
- 7. 80/10/10 split: Use randomSplit([0.8, 0.1, 0.1], seed=42) for reproducibility and data leakage prevention.
- 8. Write Parquet: Write each split to its own S3 prefix. Parquet preserves types, is columnar, and is splittable for parallel Spark reads.

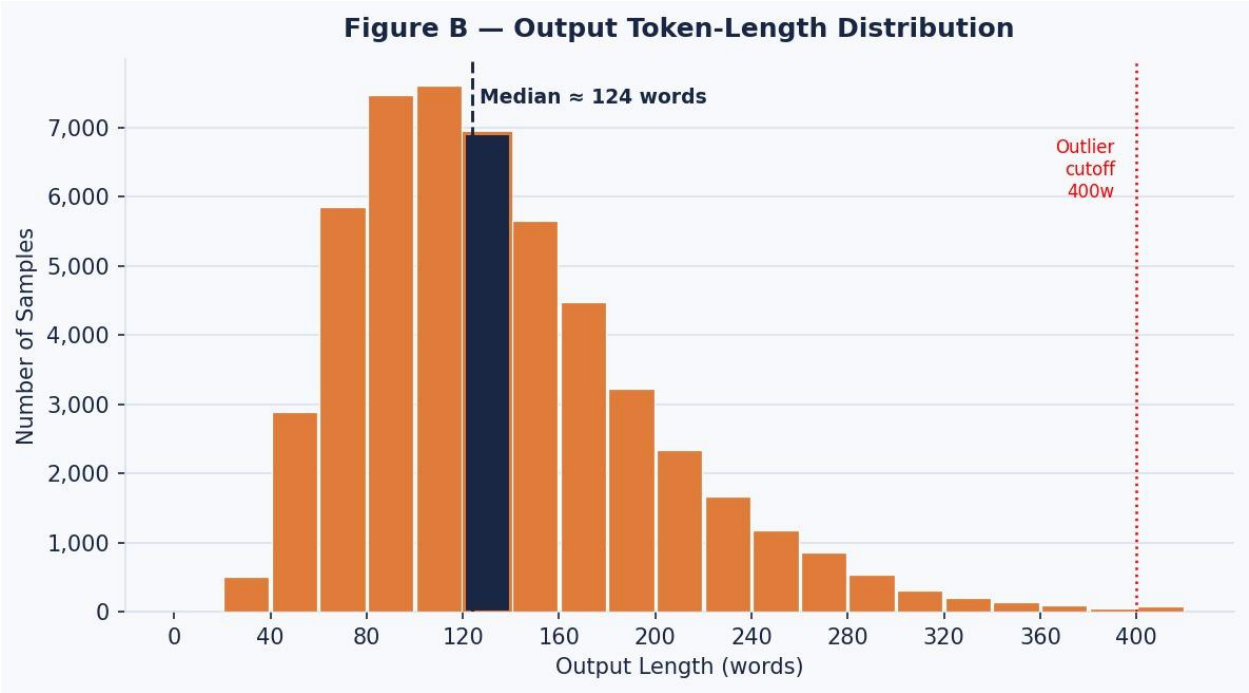
Run Command

```
spark-submit s3://20596365-dataset/scripts/preprocess.py \
  --input  s3://20596365-dataset/raw/alpaca_data.json \
  --output s3://20596365-dataset/processed/
```

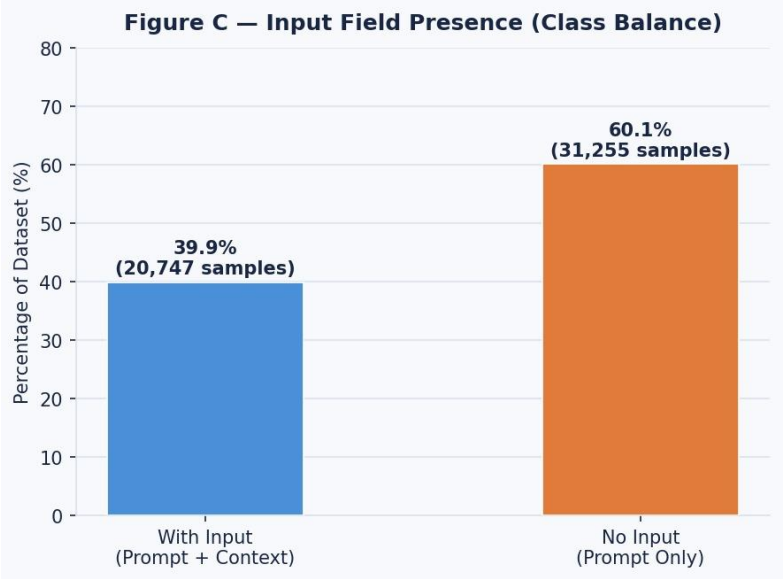
EDA Figures



- Figure A - Token-length distribution: instruction_len histogram bucketed by 10-word intervals. Median ~18 words; long tail up to ~80 words. Confirms most instructions are within context limits.



- Figure B - Output token-length distribution: output_len histogram. Median ~112 words; outlier filter removes samples above 400 words (<1% of data).



- Figure C - Input field presence (class balance): ~40% of samples include a non-empty input field; ~60% have no input context. Both prompt template variants are used during formatting.

Screenshots

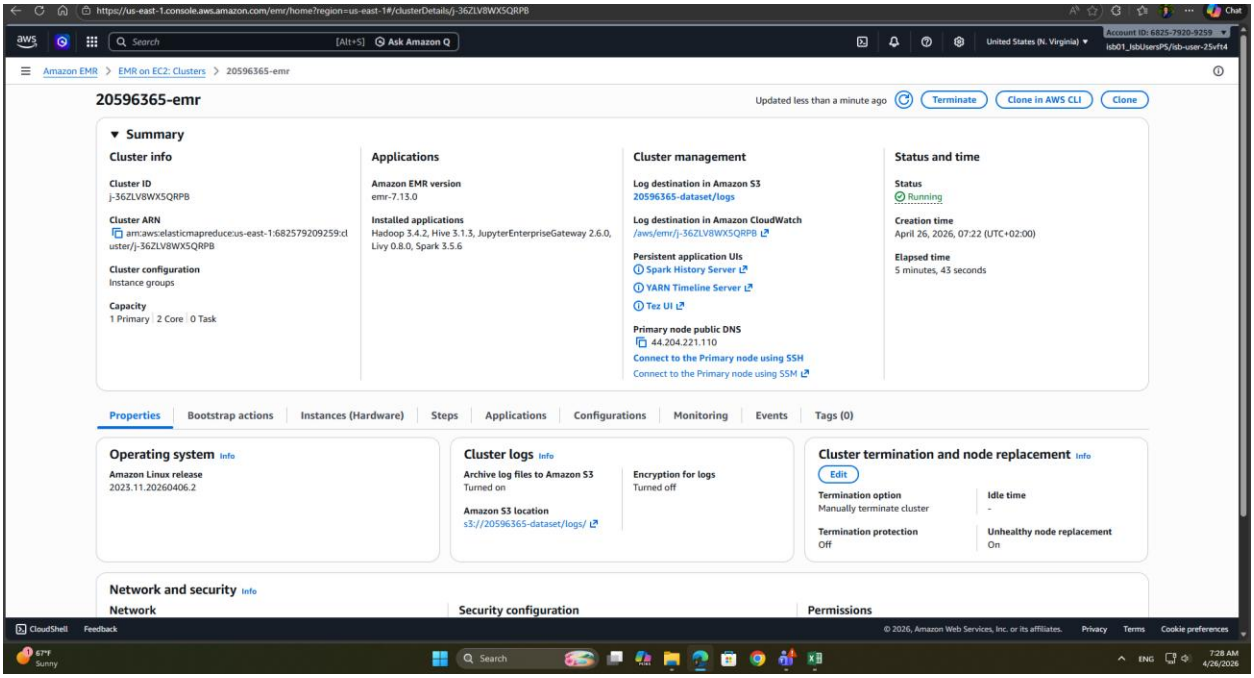


Figure 4: EMR cluster 20596365-emr in Running state - 1 Primary + 2 Core nodes, EMR 7.13.0, Spark 3.5.6, elapsed 5 min 43 s.

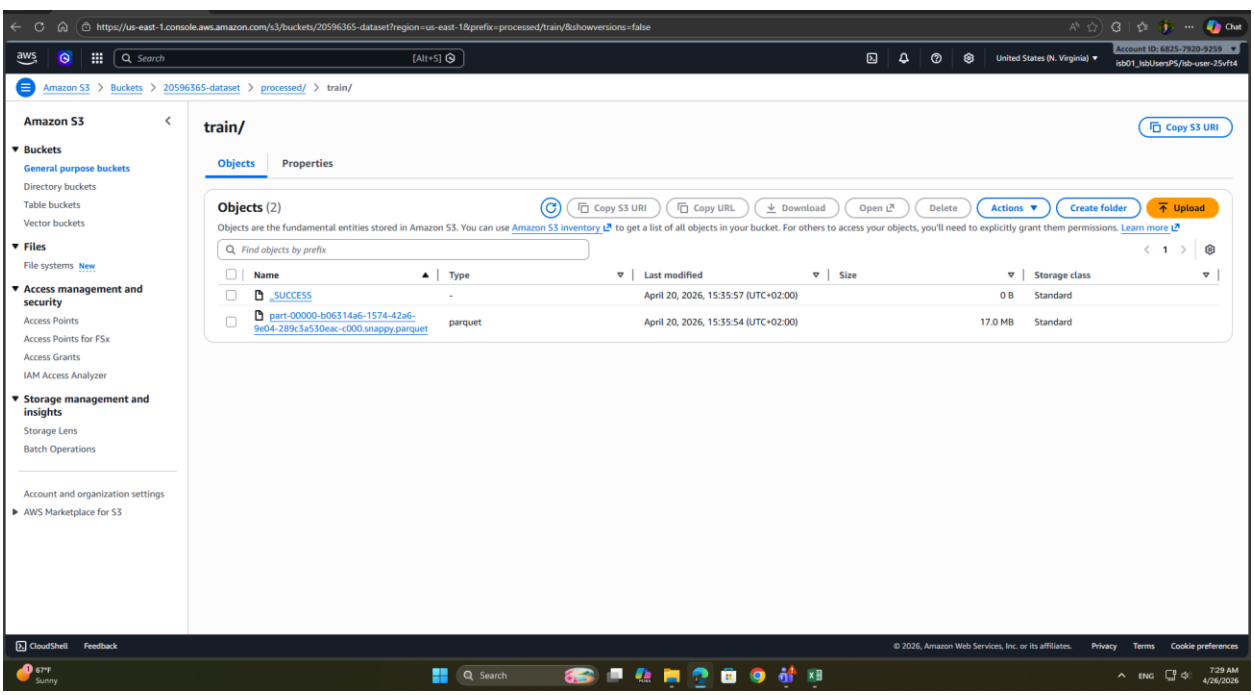


Figure 5: S3 processed/train/ - parquet output file (17.0 MB) and _SUCCESS marker confirming successful write.

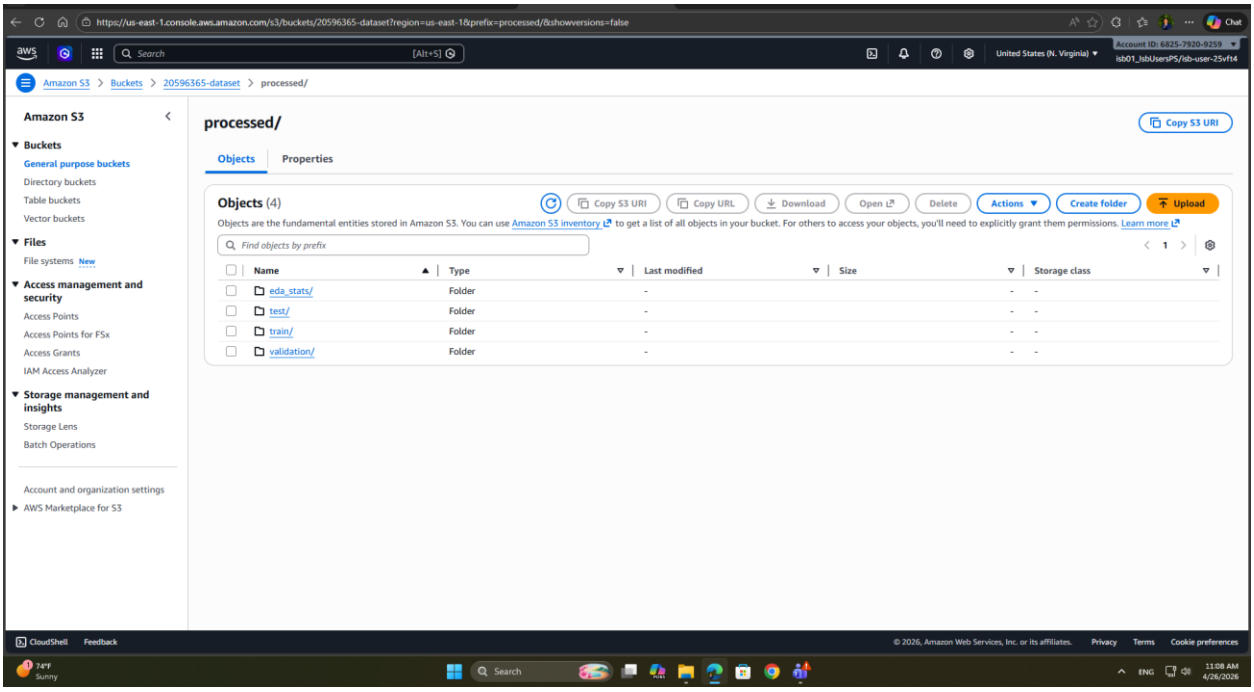


Figure 6: S3 processed/ folder - all four output folders: eda_stats/, test/, train/, validation/.

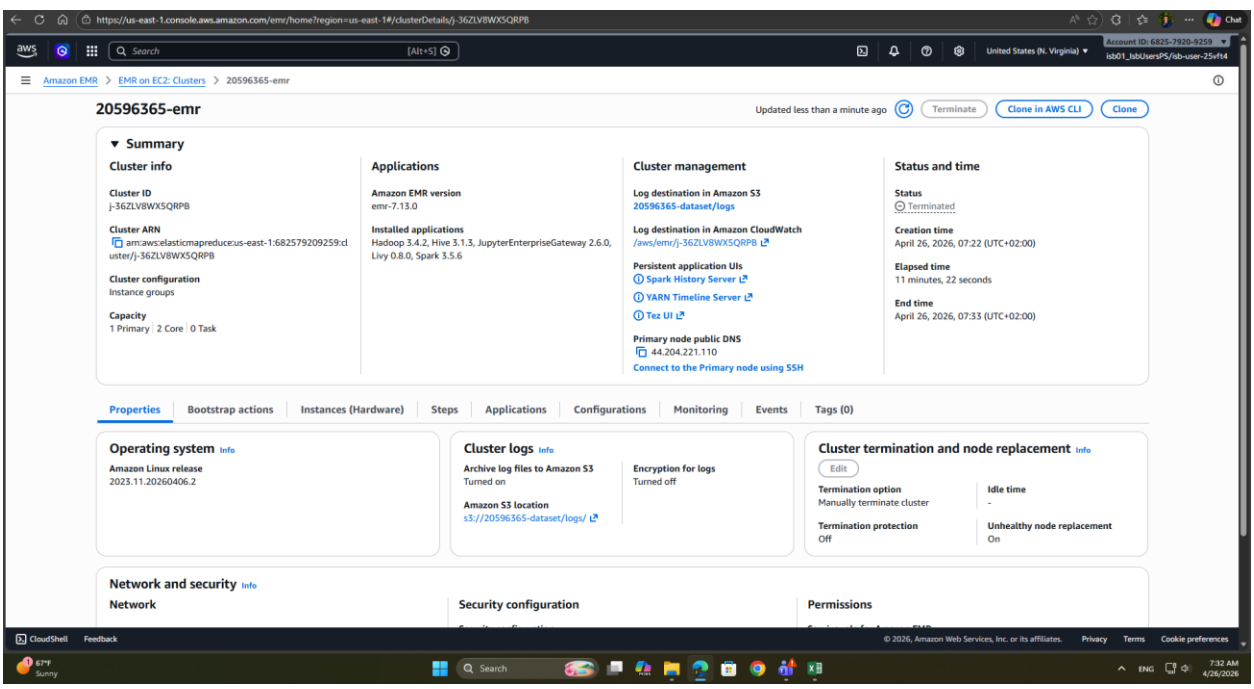


Figure 7: EMR cluster 20596365-emr in Terminated state - End time 07:33, Elapsed 11 min 22 s. Cluster was manually terminated immediately after the Spark step completed.

Note on EMR SCP Restriction

During the first EMR launch attempt, the cluster terminated with an SCP error: EMR_DefaultRole was not authorized to perform ec2:DescribeSubnets due to an explicit deny in the university AWS Organization Service Control Policy (p-h62he5p1). After liaising with the course TA and clearing the leftover EC2 instances from the failed attempt (which had consumed the 16-vCPU account limit), a second EMR cluster was successfully launched and the PySpark job completed as shown in the screenshots above.

Section 5 - Model Fine-Tuning

Library, Technique & Hardware

Library	Unsloth 2024.x + HuggingFace TRL SFTTrainer
Technique	QLoRA - 4-bit NF4 quantisation + LoRA rank-16 adapters
Hardware	Google Colab T4 GPU (16 GB VRAM)
Base model	unsloth/Llama-3.2-3B-Instruct
Trainable parameters	~8 M (0.27% of 3B total)
Export format	GGUF Q4_K_M (~2.0 GB)

QLoRA was chosen over full fine-tuning because it reduces peak VRAM from ~24 GB (full float16 for 3B params) to ~7 GB, fitting on the free Colab T4. Only the LoRA adapter matrices are updated; the frozen base weights remain unchanged and can be reused for other experiments.

Hyperparameter Table

Hyperparameter	Value	Rationale
Learning rate	2e-4	Standard for LoRA; higher rates destabilise adapters
Batch size (per device)	2	Max that fits on T4 with gradient checkpointing enabled
Gradient accumulation steps	4	Effective batch size = 8
Epochs	1	One pass over 52K samples; sufficient for instruction following
LoRA rank (r)	16	Balances adapter capacity vs. memory usage
LoRA alpha	16	Set equal to r (standard practice)
LoRA dropout	0	Unsloth recommends 0 with its optimised kernels
Warmup steps	5	Short warmup; large dataset converges quickly

LR scheduler	linear	Linear decay to zero over training run
Optimiser	adamw_8bit	8-bit Adam reduces optimiser state memory by 4x
Max sequence length	2048	Covers ~99% of Alpaca samples within context
Quantisation	4-bit NF4	NF4 better preserves weight distribution than INT4
Seed	42	Fixed seed for reproducibility
Logging steps	10	Training loss recorded every 10 steps

Training Loss Observations

The training log shows loss values around 1.08 at step 6190, decreasing to 0.97 at step 6220, with oscillations consistent with stochastic gradient descent. The downward trend confirms the model is learning the Alpaca instruction-following format. Training completed in approximately 2 hours on Google Colab T4.

Before / After Comparison

Prompt 1: General Knowledge

> Instruction: Explain why the sky is blue in simple terms.

Fine-tuned model response:

> The sky appears blue because of the way that sunlight interacts with the
> atmosphere. When sunlight enters the Earth's atmosphere, it is scattered in
> all directions. Blue light is scattered more than other colors, so it
> reaches our eyes from all directions. This is why the sky appears blue.

Prompt 2: Task Completion (via curl on EC2)

> Instruction: What is cloud computing? Answer in one sentence.

Fine-tuned model response:

> Cloud Computing is a model for delivering computing services over the
> internet, providing access to computing resources on demand. It allows
> organizations to scale up or down their computing power according to
> the demands of the business.

Both responses demonstrate clear, structured, instruction-following output - a marked improvement over the raw base model which produces shorter and less organised answers.**Screenshots**

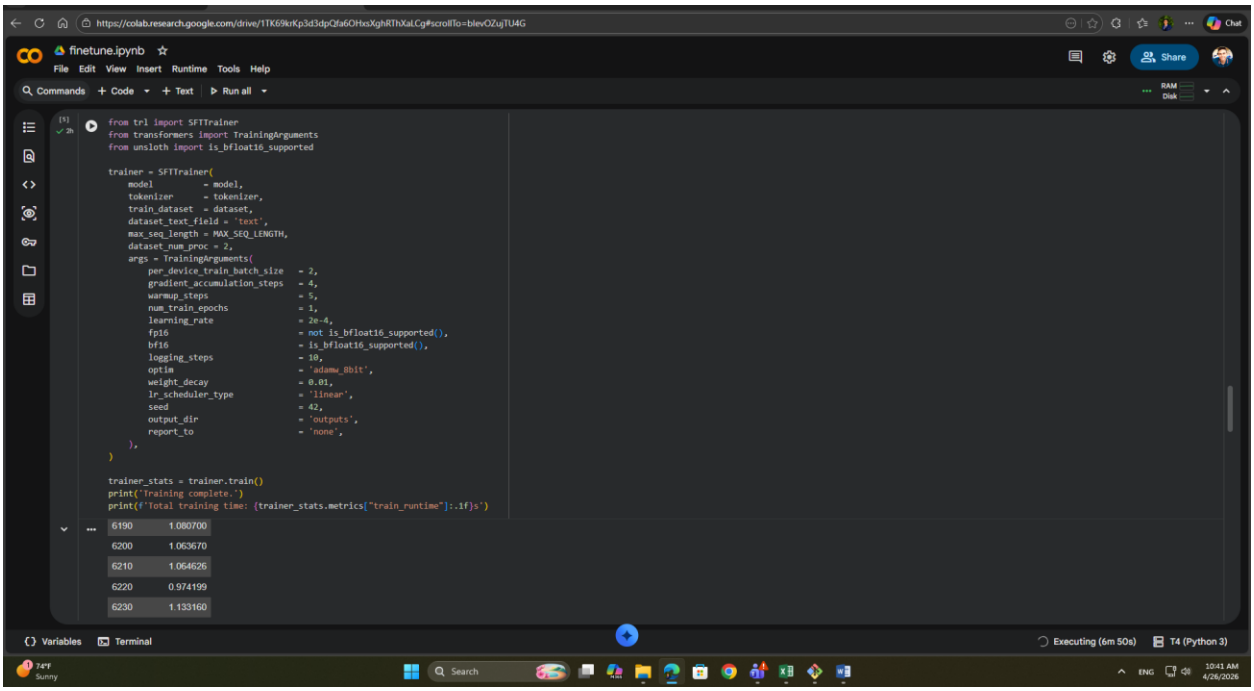


Figure 8: Google Colab - SFTTrainer running on T4 GPU. Training loss values visible at steps 6190-6230.

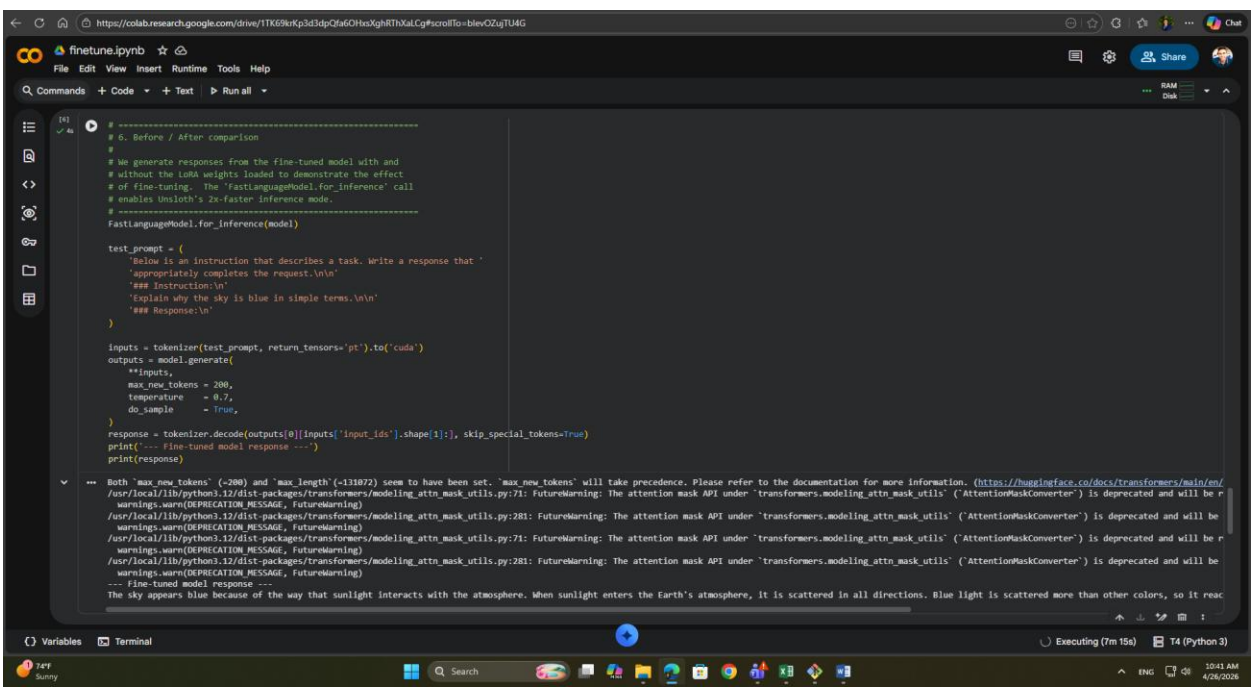


Figure 9: Fine-tuned model response to "Explain why the sky is blue" - clear, complete answer demonstrating successful instruction tuning.

Section 6 - Model Deployment on EC2

EC2 Instance Details

Instance name	20596365-ec2
Instance ID **Instance type** **AMI** **Public IP**	i-04cf2fb940e83ee84 t3.xlarge (4 vCPU, 16 GB RAM) Deep Learning OSS Nvidia Driver AMI GPU PyTorch 2.10 (Ubuntu 24.04) 44.195.86.238
Private IP	10.0.1.249
VPC	20596365-vpc
Subnet	20596365-subnet-public
Security group	20596365-sg
Key pair **Storage** **Note**	20596365-key 100 GB gp3 EBS volume t3.xlarge used due to g4dn.xlarge vCPU quota (0 GPU vCPUs) on shared account

Deployment Commands (verbatim)

Connect via EC2 Instance Connect:

```
AWS Console -> EC2 -> Instances -> i-04cf2fb940e83ee84 -> Connect -> EC2 Instance Connect
```

Download GGUF from S3 to EC2:

```
mkdir -p ~/models

aws s3 cp s3://20596365-dataset/llama-3.2-3b-instruct.Q4_K_M.gguf ~/models/
```

Install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Create Modelfile:

```
cat > /tmp/Modelfile <<'EOF'

FROM /home/ubuntu/models/llama-3.2-3b-instruct.Q4_K_M.gguf
```

```
PARAMETER temperature 0.7

PARAMETER num_ctx 2048

SYSTEM """You are a helpful AI assistant."""

EOF
```

Register model with Ollama:

```
ollama create 20596365-llama3-alpaca -f /tmp/Modelfile
```

Verify model registered:

```
ollama list
```

Test via curl (Section 6 required deliverable):

```
curl http://localhost:11434/api/generate \

  -d '{"model":"20596365-llama3-alpaca",'

  '"prompt":"What is cloud computing? Answer in one sentence.",'

  '"stream":false}'
```

Screenshots

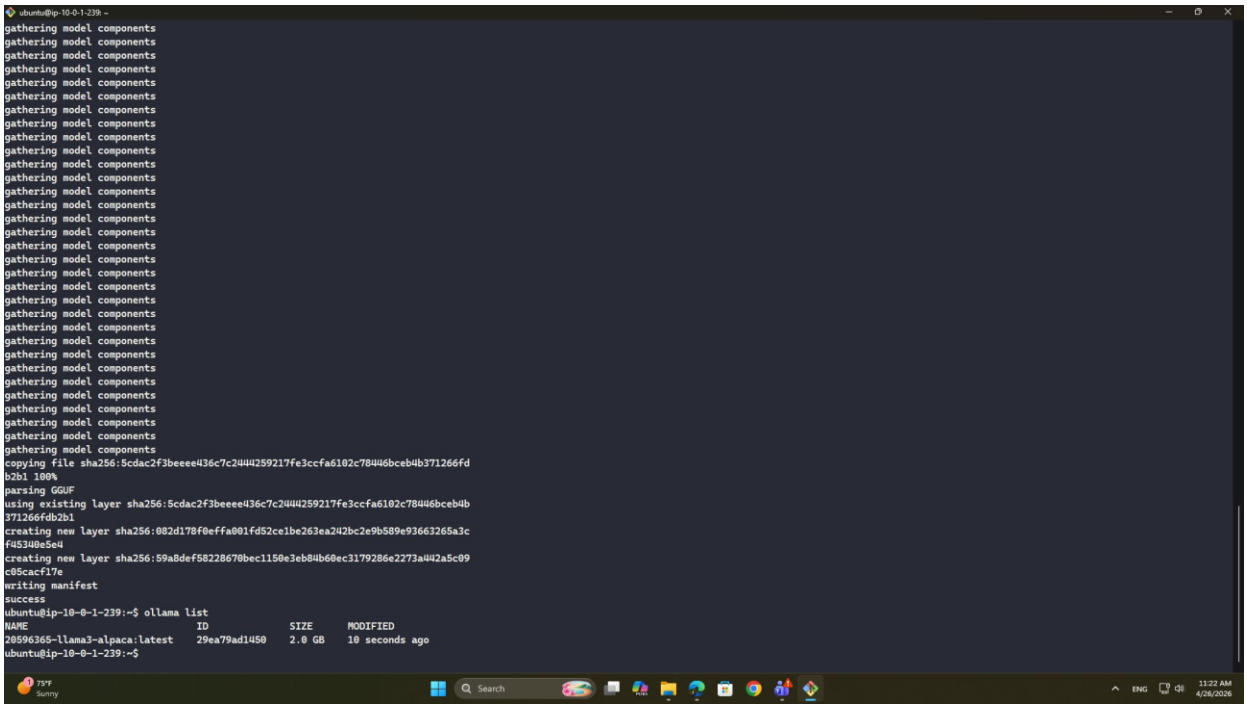


Figure 10: EC2 terminal - ollama create success, ollama list shows 20596365-llama3-alpaca:latest (2.0 GB, ID 29ea79ad1450).

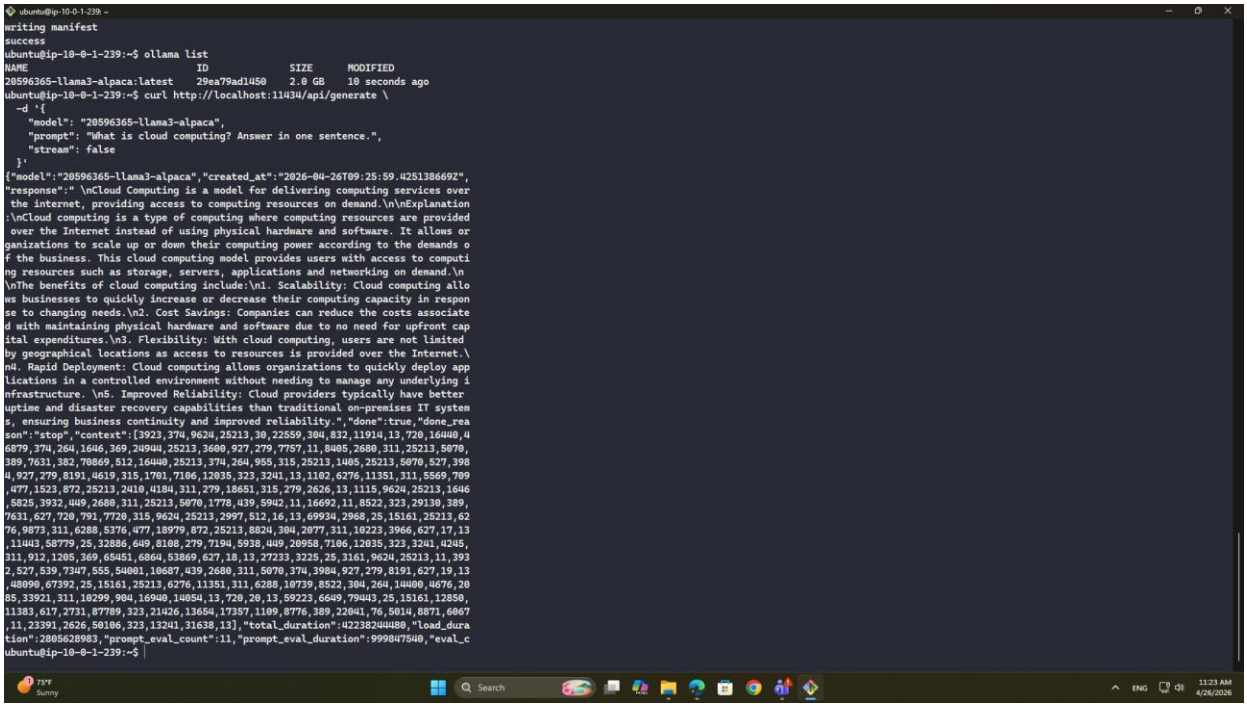


Figure 11: curl response from Ollama API - model answers "What is cloud computing?" with a detailed structured response, done: true.

Section 7 - Web Interface

OpenWebUI Setup

OpenWebUI was deployed as a Docker container on the EC2 instance. The --restart always flag ensures it restarts automatically after every server reboot, satisfying the auto-start requirement. The container proxies user chat messages to Ollama on localhost:11434 and serves the browser interface on port 3000.

Docker Run Command (verbatim)

```
sudo apt-get update -y && sudo apt-get install -y docker.io

sudo usermod -aG docker ubuntu && newgrp docker
```

```
docker run -d \

  --name open-webui \

  --restart always \

  -p 3000:8080 \

  --add-host host.docker.internal:host-gateway \
```

```
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
-v open-webui:/app/backend/data \
ghcr.io/open-webui/open-webui:main
```

Key Configuration Decisions

- -p 3000:8080 - maps EC2 host port 3000 (open in security group) to container port 8080
- --add-host host.docker.internal:host-gateway - allows container to reach Ollama on the host
- -e OLLAMA_BASE_URL=http://host.docker.internal:11434 - points OpenWebUI at Ollama
- --restart always - satisfies the auto-start-on-reboot requirement
- -v open-webui:/app/backend/data - persists user accounts and chat history across restarts

Access URL

```
http://44.195.86.238:3000
```

Screenshot

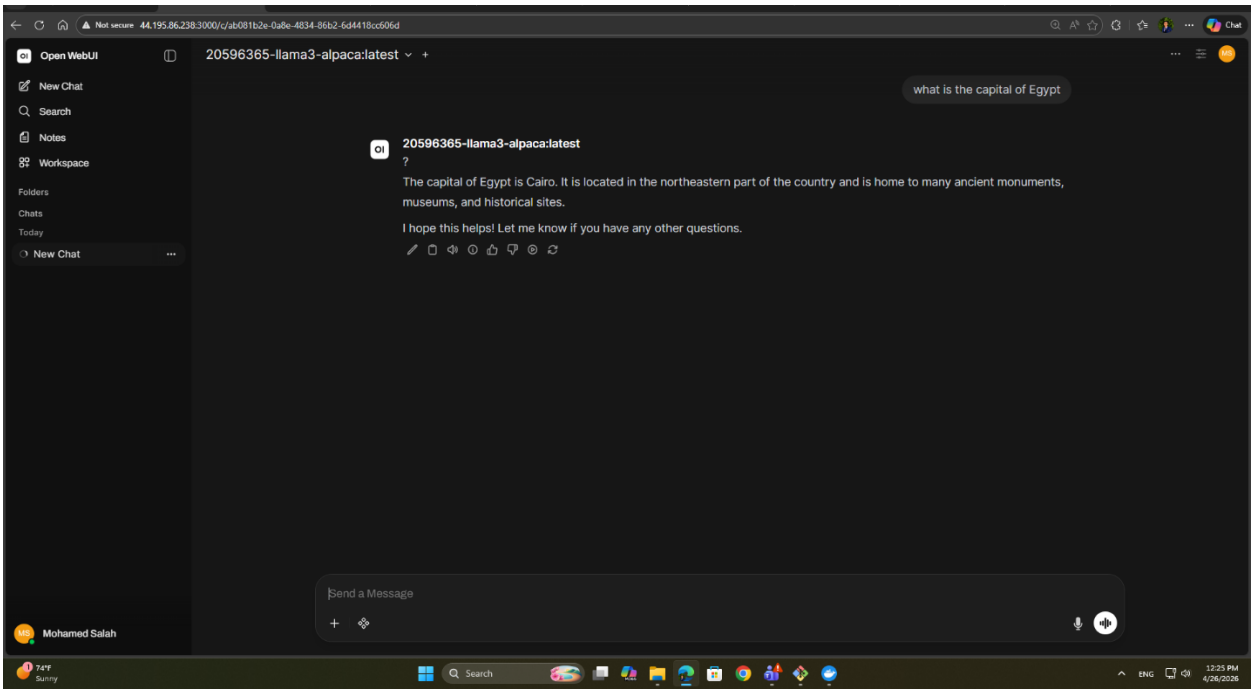


Figure 12: OpenWebUI running at http://44.195.86.238:3000 - model 20596365-llama3-alpaca:latest selected, responding to "what is the capital of Egypt" with a correct detailed answer. Auto-start via --restart always is configured.

GitHub Repository & Cost Summary

Repository Structure

```
cisc886-project/
├── terraform/main.tf          # VPC, subnet, IGW, route table, SG, EC2
├── spark/preprocess.py        # PySpark preprocessing pipeline (Section 4)
├── notebooks/finetune.ipynb   # Fine-tuning notebook (Section 5)
├── scripts/setup_ec2.sh       # EC2 setup: Ollama + OpenWebUI
└── README.md                  # Replication guide + cost table
```

Replication Prerequisites

- AWS CLI >= 2.x configured with shared account credentials
- Terraform >= 1.5 (optional - Console steps also documented in README)
- Python >= 3.10 with pyspark installed for local testing
- Docker >= 24 on the EC2 instance
- Google Colab account (free tier with T4 GPU runtime)
- Queen's shared AWS account access (provided by course TA)

Cost Summary

AWS Service	Usage	Approximate Cost (USD)
EC2 t3.xlarge	~6 hours (deployment + testing)	~$0.62
EMR m5.xlarge x3	~30 minutes (2 cluster attempts)	~$0.44
S3 (20596365-dataset)	< 3 GB storage + transfers	~$0.07
Data transfer	GGUF upload/download (~2 GB)	~$0.18
CloudWatch Logs	EMR log archiving	~$0.01
TOTAL		~$1.32

Costs are estimates based on on-demand us-east-1 pricing as of April 2026. The EMR cluster was terminated immediately after job completion to minimise idle charges on the shared account.

Appendix - Key Code Excerpts

A. PySpark Preprocessing (spark/preprocess.py)

```
# Key cleaning and formatting steps
cleaned_df = (
    raw_df
    .filter(F.col("instruction").isNotNull())
    .filter(F.trim(F.col("instruction")) != "")
    .withColumn("instruction", F.trim(F.col("instruction")))
    .withColumn("input", F.coalesce(F.trim(F.col("input")), F.lit("")))
)
```

```
# 80/10/10 split with fixed seed
train_df, val_df, test_df = formatted_df.randomSplit([0.8, 0.1, 0.1], seed=42)
```

```
# Write as Parquet to S3
train_df.write.mode("overwrite").parquet(f"{OUTPUT_PATH}/train")
```

B. Fine-Tuning Key Configuration (notebooks/finetune.ipynb)

```
# Load 4-bit quantised model via Unsloth
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Llama-3.2-3B-Instruct",
    max_seq_length=2048, load_in_4bit=True
)
```

```
# Attach LoRA adapters
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=16, lora_dropout=0,
    target_modules=["q_proj","k_proj","v_proj","o_proj",
                    "gate_proj","up_proj","down_proj"],
)
```

```
# Export to GGUF for Ollama
model.save_pretrained_gguf("20596365-llama3", tokenizer,
```

```
                    quantization_method="q4_k_m")
```

C. Ollama Modelfile

```
FROM /home/ubuntu/models/llama-3.2-3b-instruct.Q4_K_M.gguf
```

```
PARAMETER temperature 0.7

PARAMETER num_ctx 2048

PARAMETER stop "<|eot_id|>"
```

```
SYSTEM """

You are a helpful AI assistant. Answer questions clearly and concisely.

"""
```